

Atelier 5 : 500 000 livraisons sans saturer la mémoire

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Chapitre** : 5, Générateurs et décorateurs · **Version** : 2026-10-23

Avant de commencer

Ce que vous allez construire	`grand_registre.py`, un outil qui traite les 500 000 livraisons de Yonnee Express en lisant le fichier **ligne par ligne** : un générateur de livraisons, un décorateur chronomètre, une mesure de la mémoire, un tableau de bord en une seule lecture, un cache des dates, et un export partiel
Durée	🟢 Débutant : 4 h · 🔵 Standard : 3 h · 🟣 Expert : 2 h (sans les défis)
Prérequis	Avoir lu le chapitre 5 ; votre dépôt `yonnee-express` (atelier 4)
Fichiers fournis	Le script `fabriquer_grand_registre.py`, qui fabrique le grand registre (500 000 livraisons, environ 25 Mo)
Rendu	Votre dépôt GitHub `yonnee-express`, avec l'étiquette `atelier-5`

La direction veut des indicateurs sur **toute** l'activité : 500 000 livraisons sur deux ans. Le petit serveur loué par l'entreprise n'a que peu de mémoire : charger tout le registre dans une liste, comme aux ateliers 3 et 4, n'est plus acceptable. Votre mission : obtenir les mêmes réponses en ne gardant **qu'une livraison à la fois** en mémoire, et le **prouver** par des mesures.

> 📍 **Point de départ** : votre dépôt de l'atelier 4. Créez `grand_registre.py`, et deux dossiers : `donnees` et `sorties`. Copiez le tuple nommé `Livraison` de `rapports.py`.

Comment travailler

Mêmes règles que les ateliers précédents : une étape à la fois, vérifiez avant de passer à la suite, un commit par étape, une docstring par fonction.

⚠️ Le grand registre ne doit **pas** aller sur GitHub (25 Mo, et il se refabrique en quelques secondes). Ajoutez ces deux lignes à votre fichier `.gitignore` (créez-le s'il n'existe pas) :

```
donnees/registre_grand.csv
sorties/
```

Étape 1 : fabriquer le grand registre (10 min)

🎯 **Objectif** : avoir le fichier de 500 000 livraisons sur votre ordinateur.

🛠️ **A faire** : copiez `fabriquer_grand_registre.py` dans le dossier `donnees`, placez-vous dans ce dossier avec le terminal, et lancez-le.

```
cd donnees
python fabriquer_grand_registre.py
cd ..
```

✅ **Vérification** : le message `registre_grand.csv : 500 000 livraisons écrites` ; le fichier pèse environ 25 Mo. Ouvrez-le avec VS Code (pas avec Excel, très lent à ouvrir un tel fichier, et qui risque de modifier les dates en l'enregistrant) : l'en-tête est `numero;date;livreur;quartier;poids;montant;frais;duree_min`.

💡 **A comprendre** : ouvrez le script. Repérez la fonction `livraisons(nombre)` : avec quel mot-clé produit-elle les livraisons ? Pourquoi `ecrivain.writerows(livraisons(NOMBRE))` n'a-t-il jamais 500 000 dictionnaires en mémoire en même temps ?

```
git add .
git commit -m "Grand registre : script de fabrication, fichier ignoré par Git"
```

Étape 2 : le générateur de livraisons (30 min)

🎯 **Objectif** : lire le registre ligne par ligne et produire des `Livraison` propres.

🛠️ **A faire**

1. `CHEMIN = "donnees/registre_grand.csv"` en haut du fichier.
2. `lire_lignes(chemin)` : une fonction génératrice qui ouvre le fichier (en UTF-8, avec `newline=""`) et donne ses lignes avec `yield from csv.DictReader(...)`.
3. `livraisons(chemin, convertir_date=lire_date)` : une fonction génératrice qui parcourt `lire_lignes(chemin)` et produit, pour chaque ligne, une `Livraison` avec les bons types : `int` pour le numéro, le montant, les frais et la durée ; `float` pour le poids ; la date convertie par la fonction `convertir_date`, reçue en paramètre.
4. Pour l'instant, `lire_date(texte)` renvoie `datetime.strptime(texte, "%Y-%m-%d").date()` (`from datetime import date, datetime`). Elle doit être écrite **avant** `livraisons`, puisqu'elle sert de valeur par défaut.
5. Affichez les 3 premières livraisons avec `islice`.

▼ 💡 Indice : la conversion en une instruction

```
yield Livraison(int(l["numero"]), convertir_date(l["date"]), l["livreur"], l["quartier"],
                float(l["poids"]), int(l["montant"]), int(l["frais"]), int(l["duree_min"]))
```

✅ **Vérification**

```
Livraison(numero=1, date=datetime.date(2025, 4, 16), livreur='Modou', quartier='Médina',
poids=4.0, montant=143000, frais=1000, duree_min=48)
Livraison(numero=2, date=datetime.date(2026, 7, 15), livreur='Ibou', quartier='Parcelles',
poids=9.5, montant=113500, frais=2500, duree_min=45)
Livraison(numero=3, date=datetime.date(2025, 4, 11), livreur='Ibou', quartier='Plateau',
poids=0.3, montant=3500, frais=1500, duree_min=47)
```

L'affichage est **instantané** : `islice` arrête la lecture après 3 lignes.

💡 **A comprendre** : pourquoi passer la fonction de conversion des dates en **paramètre**, au lieu d'appeler `lire_date` directement ? (Réponse à l'étape 5.)

```
git add .
git commit -m "Générateur de livraisons"
```

Étape 3 : chronomètre et mémoire, liste contre générateur (40 min)

🎯 **Objectif** : prouver par des mesures que le générateur économise la mémoire.

🛠️ **A faire**

1. Le décorateur `chronometre(fonction)` du chapitre (section 7.2), avec `@wraps`, qui affiche `[nom : 1.23 s]` (deux chiffres après la virgule suffisent).
2. Deux fonctions décorées par `@chronometre`, qui calculent le total des frais :
   - `total_frais_liste(chemin)` : `list(livraisons(chemin))`, puis la somme des frais ;
   - `total_frais_generateur(chemin)` : la somme des frais directement sur `livraisons(chemin)`.
3. `memoire_maximale(fonction, *args)` : démarre `tracemalloc`, appelle `fonction(*args)`, lit le pic, arrête `tracemalloc`, et renvoie le résultat et le pic **en mégaoctets** (divisé par 1 000 000).
4. Affichez pour chacune le total (milliers séparés par une espace) et la mémoire maximale.

✅ **Vérification** (les durées dépendent de votre ordinateur ; les totaux et la mémoire, non)

```
[total_frais_liste : 12.96 s]
total_frais_liste : 1 048 561 000 F ; mémoire maximale 162.4 Mo
[total_frais_generateur : 11.29 s]
total_frais_generateur : 1 048 561 000 F ; mémoire maximale 0.1 Mo
```

⚠️ Pour séparer les milliers, formatez **d'abord** le nombre seul, puis remplacez ses virgules : `total_lisible = f"{total:,}".replace(",", " ")`. Si vous appliquez `.replace` à toute la phrase, il supprime aussi les virgules de votre texte.

💡 **A comprendre**

1. Au chapitre (section 5.1), la liste de dictionnaires montait à 326 Mo. Ici, une liste de tuples nommés : 162 Mo. Pourquoi un tuple nommé prend-il moins de place qu'un dictionnaire ?
2. Les deux versions prennent presque le même temps. Le générateur rend-il le programme plus **rapide** ? Qu'économise-t-il vraiment ?

```
git add .
git commit -m "Chronomètre et mesure de mémoire"
```

Étape 4 : le tableau de bord en une seule lecture (40 min)

🎯 **Objectif** : plusieurs indicateurs en parcourant le fichier **une seule fois**.

Lire 500 000 lignes prend environ 2 secondes. Avec un générateur, on ne peut parcourir les livraisons qu'une fois : si on veut quatre indicateurs, faut-il lire le fichier quatre fois ? Non : on les calcule **tous dans la même boucle**.

🛠️ **A faire** : `tableau_de_bord(chemin, seuil=60)`, décorée par `@chronometre`, renvoie en une seule boucle sur `livraisons(chemin)` :

Indicateur	Outil conseillé
le nombre de livraisons	un compteur `nombre += 1`
les frais par année	un `defaultdict(int)` ; l'année est `l.date.year`
le nombre de retards (plus de `seuil` minutes) par quartier	un `Counter`
la livraison la plus longue	une variable `plus_longue`, d'abord `None`, remplacée quand on trouve plus long

▼ 💡 Indice : la plus longue

```
if plus_longue is None or l.duree_min > plus_longue.duree_min:
    plus_longue = l
```

✅ **Vérification**

```
  [tableau_de_bord : 2.60 s]
  500000 livraisons ; frais par année : {2025: 579896000, 2026: 468665000}
  retards par quartier : [('Pikine', 27458), ('Parcelles', 13661), ('Yoff', 5582), ('Plateau',
1712), ('Médina', 458)]
  la plus longue : 85559 Pikine 104 min
```

Vérifiez la cohérence : 579 896 000 + 468 665 000 = 1 048 561 000, le total de l'étape 3.

💡 **A comprendre** : 2026 a moins de frais que 2025. Est-ce que l'activité baisse ? Regardez la dernière date du registre (indice : le registre s'arrête en 2026, quel mois ?).

```
git add .
git commit -m "Tableau de bord en une lecture"
```

Étape 5 : le cache des dates (35 min)

🎯 **Objectif** : accélérer une fonction lente avec `lru_cache`, et apprendre à **mesurer avant d'optimiser**.

Le registre contient 500 000 dates, mais seulement environ 660 dates **différentes** (deux ans de jours) : la même date revient des centaines de fois. C'est le cas idéal pour un cache.

🛠️ A faire

1. Gardez une version sans cache, `lire_date_lente(texte)`, identique à `lire_date`.
2. Décorez `lire_date` par `@lru_cache(maxsize=None)`.
3. `convertir_toutes_les_dates(textes, convertir_date)`, décorée par `@chronometre`, renvoie la liste des dates converties.
4. Préparez la liste des textes de dates : `textes = [l["date"] for l in lire_lignes(CHEMIN)]`. Puis, dans l'ordre : `lire_date.cache_clear()` ; la conversion avec `lire_date_lente` ; la conversion avec `lire_date` ; `print(lire_date.cache_info())` ; enfin la conversion avec `date.fromisoformat`, la fonction de Python faite pour le format `AAAA-MM-JJ`.

✅ Vérification

```
  [convertir_toutes_les_dates : 3.32 s]
  [convertir_toutes_les_dates : 0.08 s]
  CacheInfo(hits=499340, misses=660, maxsize=None, currsize=660)
  [convertir_toutes_les_dates : 0.05 s]
```

💡 A comprendre

1. Expliquez les nombres de `cache_info()` : pourquoi exactement 660 `misses` ? Que vaut `hits + misses` ?
2. Le cache rend `strptime` 40 fois plus rapide. Mais `fromisoformat`, **sans** cache, est encore plus rapide. Qu'en concluez-vous sur la bonne démarche : ajouter des caches partout, ou mesurer d'abord ?

3. Revenez à la question de l'étape 2 : grâce au paramètre `convertir_date`, comment changer la conversion utilisée par **tout** l'outil, sans modifier `livraisons` ?

```
git add .
git commit -m "Cache des dates mesuré"
```

Étape 6 : exporter les 1 000 premiers retards (25 min)

🎯 **Objectif** : un export partiel qui ne lit **que** ce dont il a besoin.

🛠️ **A faire** : `exporter_retards(chemin, sortie, combien=1000, seuil=60)`, décorée par `@chronometre` :

1. une expression génératrice des livraisons en retard ;
2. ouvrir le fichier de sortie en écriture (UTF-8, `newline=""`), un `csv.writer` avec `;` ;
3. écrire l'en-tête : `Livraison._fields` donne la liste des noms de champs ;
4. écrire les `combien` premiers retards avec `islice` ; la date doit être réécrite au format `AAAA-MM-JJ` (`l.date.isoformat()`).

Appelez-la avec `"sorties/retards_extrait.csv"`.

▼ 💡 Indice : remplacer un seul champ d'un tuple

`[*l[:1], l.date.isoformat(), *l[2:]]` : le premier champ, la date en texte, puis tous les champs à partir du troisième. On peut aussi écrire la liste des huit champs à la main.

✅ **Vérification**

```
  [exporter_retards : 0.05 s]
```

Le fichier a 1 001 lignes (l'en-tête et 1 000 retards). Sa deuxième ligne : `5;2026-06-10;Awa;Yoff;11.8;88000;2500;61`. Sa dernière : la livraison n° 9821.

💡 **A comprendre** : le tableau de bord lisait le fichier en 2,6 s, l'export en 0,05 s. Le 1 000e retard est la livraison n° 9821 : combien de lignes du fichier ont été lues ? Quel pourcentage du fichier ? Que se serait-il passé avec une **liste** des retards à la place de l'expression génératrice ?

```
git add .
git commit -m "Export partiel des retards"
```

Étape 7 : la démonstration (15 min)

🛠️ **A faire** : une fonction `main()` qui enchaîne les étapes 2 à 6 avec un titre par étape, puis `if __name__ == "__main__": main()`.

✅ **Vérification** : `python grand_registre.py` affiche tout sans erreur en moins d'une minute environ.

Puis étiquetez :

```
git add .
git commit -m "Démonstration du grand registre"
git tag atelier-5
git push
git push origin atelier-5
```

🏆 Défis (facultatifs)

1. **Le générateur de lots.** Écrivez `par_lots(iterable, taille)`, une fonction génératrice qui donne des listes de `taille` éléments (la dernière peut être plus courte). Avec `par_lots(livraisons(CHEMIN), 50_000)`, affichez le total des frais de chaque lot de 50 000 livraisons. Indice : `islice` dans une boucle `while True`, et s'arrêter quand le lot est vide.
2. **Le décorateur de mémoire.** Transformez `memoire_maximale` en décorateur `@mesurer_memoire`, qui affiche la mémoire maximale à chaque appel, comme `chronometre` affiche la durée. Empilez-le avec `@chronometre` : que remarquez-vous sur la durée affichée selon l'ordre des deux décorateurs ?
3. **Le mois record.** Sans liste, en une lecture : quel mois (année et mois) a le plus de livraisons ? Indice : un `Counter` sur `(l.date.year, l.date.month)`.
4. 🟣 **Le pipeline réglable.** Réécrivez le tableau de bord pour qu'il accepte des conditions de l'atelier 4 (`dans_quartiers`, `plus_lourd_que`…) : `tableau_de_bord(CHEMIN, dans_quartiers("Yoff"), plus_lourd_que(10))`. Le filtrage doit rester paresseux.

Grille d'autoévaluation

Je sais…	Oui	Pas encore
écrire une fonction génératrice avec `yield` et `yield from`	☐	☐
expliquer pourquoi un générateur économise la mémoire, et le mesurer	☐	☐
calculer plusieurs indicateurs en une seule lecture	☐	☐
écrire un décorateur avec `@wraps`	☐	☐
utiliser `lru_cache` quand il est utile, et mesurer avant d'optimiser	☐	☐
arrêter une lecture au bon moment avec `islice`	☐	☐